90% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Detection Groups

17 AI-generated only 90%
Likely AI-generated text from a large-language model.

0 AI-generated text that was AI-paraphrased 0%
Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?
The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?
Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.

Policy-Guided Virtual Mentoring for Introductory Programming: A Pilot Study

Anonymized for double-blind review

Abstract

Introductory programming is difficult for many first-year students because small syntax errors, weak debugging habits, and incomplete conceptual understanding often appear at the same time. In large classes, instructors and teaching assistants cannot always respond while the student is still working through the problem. Automated tools can help, but their feedback is often limited to compiler messages, test results, or generic hints. This paper describes a real-time virtual mentor for introductory programming that gives short, policy-guided feedback during active coding. The mentor combines code analysis, a lightweight student state, and a feedback policy that chooses among four actions: guiding question, conceptual hint, code highlight, and no feedback. A language model may be used to phrase the final message, but the decision about when and how to intervene is handled by a separate policy layer. The prototype was examined through 78 event-level interaction logs. The mean end-to-end latency was 740 ms, with a P95 latency of 800 ms. The event-level success rate was 70.0%, and the median time-to-fix was 2.7 s. These results do not prove learning gains, but they show that policy-guided mentoring can operate within an interactive time range and can provide a practical basis for classroom-scale evaluation.

Keywords: virtual mentor, introductory programming, CS1, programming education, real-time feedback, student modeling, pedagogical policy

1 Introduction

Many students meet programming for the first time in a CS1 course. The first weeks are rarely difficult because of one isolated concept. More often, students have to deal with several things at once: unfamiliar syntax, compiler errors, problem decomposition, test cases, and the need to reason about code that does not yet work. When feedback arrives late, beginners may keep repeating the same mistake without understanding what actually went wrong.

This problem is especially visible in large classes. Instructors and teaching assistants can explain mistakes well, but they cannot always sit next to every student at the moment when the student is stuck. Automated assessment systems improve scalability through compiler messages, unit tests, and static checks, yet this feedback is often binary. It can tell a student that a solution is wrong, but it may not explain what kind of thinking is needed next [1, 2].

Recent AI-based teaching assistants show that more flexible support is possible. Systems such as CodeAid, Iris, and the CS50 AI assistant demonstrate that students can receive natural-language help at scale [3–5]. At the same time, programming education research also points to clear risks. Students may ask for answers too quickly, accept inaccurate explanations, or use the assistant as a shortcut instead of learning how to debug [6–8]. For this reason, an educational assistant should not only generate fluent text. It should also decide when feedback is useful, how explicit it should be, and when silence is better.

The work reported here takes that position. The virtual mentor is designed as a bounded real-time decision pipeline rather than as an open-ended chat assistant. The system observes code edits, extracts simple signals about the current programming state, updates a compact student profile, and selects a feedback action under pedagogical constraints. The final message can be

generated from templates or by a language model, but the language model is not responsible for the core mentoring decision.

The study is guided by two research questions:

- **RQ1:** Can a modular virtual mentor provide feedback within an interactive latency range during introductory programming tasks?
- **RQ2:** What do pilot event logs show about the short-term resolution patterns associated with different feedback actions?

The contribution is a compact architecture and pilot evaluation for policy-guided programming feedback. The paper does not claim that the current prototype improves final course grades or long-term retention. Its purpose is more limited: to show how real-time code analysis, student-state tracking, and controlled feedback selection can be combined into a working mentoring loop that is suitable for later classroom studies.

2 Literature Review

2.1 Intelligent tutoring systems for programming

Intelligent tutoring systems personalize support by modeling learner knowledge and adapting hints, exercises, or feedback. In programming education, this idea is attractive because students often make repeated errors that are visible in their code. Recent systems also tend to frame AI as support for teachers rather than as a replacement for them. For example, ProgTutor combines adaptive sequencing with automated evaluation while keeping the instructor in the learning process [9].

A limitation of many tutoring systems is the interaction point. Some systems intervene after submission, after a completed step, or inside a fixed exercise flow. That can be useful, but it does not always match the moment when a beginner needs help most: while the code is still being written and the student is deciding what to try next.

2.2 Automated feedback for programming assignments

Automated feedback has a long history in programming education. Unit tests, compiler diagnostics, static analysis, and repair suggestions can reduce grading effort and give students faster responses. Reviews of automated feedback systems show that these tools are valuable, but they also report recurring problems: feedback may be too general, too focused on correctness, or hard to adapt to a student's current misconception [1, 2].

For novice programmers, the same error message can mean different things. A missing semicolon, a wrong loop boundary, or a failed test may be a simple slip for one student and a sign of a deeper misunderstanding for another. This is why feedback needs context. The mentor in this study uses a small student state to track repeated error categories and recent interventions instead of treating every code state as isolated.

2.3 LLM-based programming assistants

Large language models make it easier to produce natural-language explanations, examples, and hints. This is useful in CS1 because students often need an explanation in ordinary language, not only a compiler message. CodeAid and Iris are examples of systems that place constraints around AI support so that students receive help without simply being given full solutions [3, 4]. Large-scale work on CS50 also shows that AI assistance can be deployed with low marginal cost, but it requires continuous monitoring and improvement [5].

The main risk is that fluency can hide weak pedagogy. A response can sound helpful while giving away too much, using a misleading explanation, or encouraging students to ask for the next step repeatedly. Prior studies on student help-seeking with LLM-powered programming assistants show that how students use the tool matters as much as whether the tool is available [6]. This supports the need for an explicit policy layer that controls the timing and detail of feedback.

2.4 Positioning of this study

The virtual mentor sits between automated feedback tools and conversational AI assistants. Like automated tools, it uses code analysis and structured signals. Like AI assistants, it can produce short natural-language feedback. The difference is that the feedback action is selected before any final message is written. This separation makes the system easier to inspect and gives instructors a clearer way to adjust the intended mentoring behavior.

Table 1 summarizes how the prototype is positioned against representative programming feedback systems.

Table 1: Positioning of the virtual mentor against representative systems

System	Core technology	Main feedback mode	Reported focus
AIF 3.0 [10]	Hybrid data-driven and expert-based approach	Progress indicators and subgoal-oriented feedback	Subgoal detection accuracy
CodeAid [3]	GPT-3.5-Turbo with classroom constraints	Pseudocode-oriented explanations and guided help	Correctness and classroom fit
Verifix [1]	Program analysis	Verified repair suggestions	Feedback quality
DeepFix [1]	Neural error-correction model	Syntax error correction	Error repair rate
Virtual mentor in this study	Analyzer, student state, policy layer, optional language model	In-process feedback through guiding questions, hints, highlights, or no feedback	Pilot latency and event-level resolution

3 Methodology

3.1 Research design

This work uses a prototype-based design and pilot evaluation. The goal was not to run a full classroom experiment at this stage, but to check whether the mentoring loop is technically and pedagogically plausible. The prototype was evaluated using event-level interaction logs produced during representative introductory programming sessions.

Each event corresponds to a code state submitted to the mentor pipeline. The log records analyzer output, selected feedback action, latency measurements, and whether the related issue was resolved after the intervention. This type of data is useful for testing responsiveness and immediate resolution patterns. It is not enough to make claims about course achievement, retention, or transfer.

3.2 System architecture

The mentor is organized as four separate components: analyzer, profile manager, policy layer, and feedback realization. The separation is important because it keeps the mentoring decision visible. Instead of sending the student's code directly to a conversational assistant and asking for help, the system first decides what type of help is appropriate.

In practice, the workflow is simple. A student edits code, the analyzer produces a short description of the current state, the profile manager updates the student's recent history, and

Table 2: Main components of the mentoring pipeline

Component	Role in the prototype
Analyzer	Checks the current code state and extracts signals such as compile status, novice error category, suspicious code region, and task progress.
Profile manager	Keeps a lightweight student state, including repeated error categories and recent feedback history.
Policy layer	Selects one feedback action under pedagogical constraints. It decides whether to ask a guiding question, give a conceptual hint, highlight a code region, or stay silent.
Feedback realization	Turns the selected action into the final student-facing response. This can be template-based or language-model-assisted, but it must follow the selected action type.

the policy layer selects an action. The feedback realization step then produces a short message or interface action. If the policy selects `NO_FEEDBACK`, the system does not interrupt the student.

3.3 Feedback actions

The prototype uses four feedback actions:

- `GUIDING_QUESTION`: a question that pushes the student to reason about the next step without giving the answer.
- `CONCEPTUAL_HINT`: a short explanation of the relevant concept or likely misconception.
- `CODE_HIGHLIGHT`: a pointer to a suspicious code region, with little or no explanation.
- `NO_FEEDBACK`: no intervention, used when immediate feedback is not necessary or may interrupt productive work.

The main rule is that the mentor should support debugging and reasoning without disclosing a complete solution. This is a small design choice, but it changes the role of the system. The mentor is not meant to be a code generator for beginners. It is meant to behave more like a careful teaching assistant who gives just enough help for the student to continue.

3.4 Policy model

The current prototype includes a rule-based policy and a learned policy extension. Both use the same feature interface, so they can be compared later without changing the rest of the system. The feature set includes error type, severity, compile status, test outcomes, recurrence counts, attempt number, previous feedback action, previous feedback success, suspicious-region presence, code length, and cumulative feedback count in the session.

The learned policy was implemented as a multi-class Random Forest classifier using `scikit-learn`. It predicts one of the four feedback actions. The model was configured with 200 trees, `max_depth=8`, `min_samples_leaf=2`, `class_weight="balanced_subsample"`, and `random_state=42`.

Training data for the learned policy was bootstrapped from a public programming-problem corpus with questions, accepted solutions, and test cases. The accepted solutions were used to synthesize novice-like error states and weak policy labels. This is useful for building the first policy pipeline, but it is not the same as having authentic tutor annotations from a classroom. For that reason, the learned policy is treated as an exploratory extension, not as a validated pedagogical model.

3.5 Evaluation data and metrics

The pilot evaluation uses 78 event-level interaction logs. Each event is represented by

$$x_i = (e_i, c_i, t_i^{analysis}, t_i^{policy}, t_i^{feedback}, a_i, s_i),$$

where e_i is the error category, c_i is the recurrence count, $t_i^{analysis}$, t_i^{policy}, and $t_i^{feedback}$ are latency components, a_i is the selected feedback action, and s_i is the event outcome indicator.

Two metric groups were used. The first group measures responsiveness through end-to-end latency and component latency. Total latency is defined as

$$T_{total} = T_{analysis} + T_{policy} + T_{feedback} + T_{overhead}.$$

The second group uses event-level effectiveness proxies: event success rate and time-to-fix. An event is counted as successful when the issue linked to the selected intervention is resolved after the feedback action. These metrics are useful for a pilot, but they should not be read as direct learning-outcome measures.

4 Results and Discussion

4.1 Responsiveness

Table 3 summarizes the latency results. The prototype stayed within an interactive range in the pilot logs. The mean end-to-end latency was 740 ms, and the P95 latency was 800 ms. Most of the time was spent in the analysis stage, while policy selection and feedback realization were small parts of the total delay.

Table 3: Pilot responsiveness results from event-level logs

Metric	Value
Events	78
Mean end-to-end latency	740 ms
P95 latency	800 ms
Mean analysis latency	618 ms
Mean policy latency	26 ms
Mean feedback realization latency	12 ms
Mean platform overhead	85 ms

These numbers matter because real-time educational support has a narrow tolerance for delay. If the response arrives too late, the student may already have changed the code, lost the thread of the problem, or started guessing. In this pilot, the mentor was fast enough to fit into the coding flow. The result also supports the architectural decision to keep policy selection lightweight.

4.2 Event-level resolution

Across the 78 events, the overall event-level success rate was 70.0%. The median time-to-fix was 2.7 s, and the P95 time-to-fix was 3.6 s. Table 4 gives the breakdown by feedback action.

The strongest immediate resolution rates were observed for code highlights and conceptual hints. This is not surprising. When a beginner is close to a solution, pointing to the relevant region or naming the concept can be enough to help the student continue. Guiding questions had a lower short-term resolution rate. That does not automatically make them worse. A guiding

Table 4: Event-level resolution by feedback action

Action	Events	Success rate
CODE_HIGHLIGHT	23	82.6%
CONCEPTUAL_HINT	26	80.8%
GUIDING_QUESTION	21	57.0%
NO_FEEDBACK	8	25.0%

question may slow down immediate fixing but still support reasoning, especially when the goal is to avoid giving away the implementation step.

The NO_FEEDBACK action had the lowest resolution rate in this trace. This should be interpreted carefully. In a policy-guided mentor, silence is not always a failure. Sometimes the system should avoid interrupting the student, especially when the evidence is weak or the student appears to be making progress. A better future evaluation should examine not only whether the next issue was fixed, but also whether the student's later attempts became more independent.

4.3 Pedagogical interpretation

The pilot results support the basic idea that feedback type should be treated as a design decision. A system that always explains everything may be helpful in the short term, but it can reduce productive struggle. A system that only reports errors may be fast, but it may leave beginners without enough direction. The virtual mentor tries to occupy the middle ground: it gives feedback, but the amount of help is controlled.

The architecture also makes the system easier to adjust for instructors. If a course wants less explicit feedback during graded work, the policy can favor guiding questions or no feedback. If the goal is early practice, the policy can allow more conceptual hints. This is harder to control when the whole mentoring behavior is hidden inside a single prompt to a conversational model.

5 Study Limitations

The study has several limitations. First, the evaluation is based on 78 pilot events, not on a full classroom deployment. The results should therefore be treated as feasibility evidence. Second, the success measures are event-level proxies. They show whether a local issue was resolved after feedback, but they do not show learning gains, retention, transfer, or final course performance. Third, the learned policy was trained with weak labels derived from a public programming-problem corpus, not from expert-labeled classroom mentoring data. This is acceptable for prototyping the pipeline, but it is not enough to claim that the learned policy is pedagogically optimal.

Another limitation is the lightweight student model. It tracks repeated errors and recent feedback history, but it does not yet model deeper learning trajectories. A richer model could improve personalization, but it would also require more data, more careful privacy handling, and more validation.

6 Future Research Directions

Future work should move from prototype-level evaluation to classroom evaluation. A larger study should compare the mentor with non-personalized automated feedback, a rule-only policy, and ordinary post-submission feedback. It should also include student-level measures, such as debugging confidence, task completion, delayed assessment, and perceived usefulness.

The learned policy should be retrained and validated using authentic student–mentor interaction traces. Ideally, some actions should be reviewed by instructors or teaching assistants so that the model learns from pedagogical judgments, not only from weak outcome labels. Future versions should also let instructors tune the feedback policy for different course phases, such as practice sessions, labs, and graded assignments.

7 Conclusion

This paper presented a policy-guided virtual mentor for introductory programming. The mentor separates code analysis, student-state tracking, feedback action selection, and final message realization. This separation keeps the system responsive and gives instructors a clearer way to control how much help students receive.

In a pilot evaluation with 78 event-level logs, the prototype achieved a mean end-to-end latency of 740 ms and a P95 latency of 800 ms. The overall event-level success rate was 70.0%, with a median time-to-fix of 2.7 s. These results are preliminary, but they show that real-time mentoring with explicit pedagogical constraints is technically feasible and worth testing in a larger CS1 classroom setting.

References

[1] S. S. Kumar, M. A. Lones, M. Maarek, and H. Zantout, "Navigating the landscape of automated feedback generation techniques for programming exercises," *ACM Trans. Comput. Educ.*, vol. 25, no. 4, Oct. 2025. [Online]. Available: https://doi.org/10.1145/3764593

[2] M. Messer, N. C. C. Brown, M. Kölling, and M. Shi, "Automated grading and feedback tools for programming education: A systematic review," *ACM Trans. Comput. Educ.*, vol. 24, no. 1, Feb. 2024. [Online]. Available: https://doi.org/10.1145/3636515

[3] M. Kazemitabaar, R. Ye, X. Wang, A. Z. Henley, P. Denny, M. Craig, and T. Grossman, "Codeaid: Evaluating a classroom deployment of an llm-based programming assistant that balances student and educator needs," 2024. [Online]. Available: https://doi.org/10.1145/3613904.3642773

[4] P. Bassner, E. Frankford, and S. Krusche, "Iris: An ai-driven virtual tutor for computer science education," pp. 394–400, 2024. [Online]. Available: https://doi.org/10.1145/3649217.3653543

[5] R. Liu, J. Zhao, B. Xu, C. Perez, Y. Zhukovets, and D. J. Malan, "Improving ai in cs50: Leveraging human feedback for better learning," pp. 715–721, 2025. [Online]. Available: https://doi.org/10.1145/3641554.3701945

[6] B. Sheese, M. Liffiton, J. Savelka, and P. Denny, "Patterns of student help-seeking when using a large language model-powered programming assistant," pp. 49–57, 2024. [Online]. Available: https://doi.org/10.1145/3636243.3636249

[7] U. Z. Ahmed, S. Sahai, B. Leong, and A. Karkare, "Feasibility study of augmenting teaching assistants with ai for cs1 programming feedback," pp. 11–17, 2025. [Online]. Available: https://doi.org/10.1145/3641554.3701972

[8] A. F. Pereira and R. Ferreira Mello, "A systematic literature review on large language models applications in computer programming teaching evaluation process," *IEEE Access*, vol. 13, pp. 113 449–113 460, 2025. [Online]. Available: https://doi.org/10.1109/ACCESS.2025.3584060

[9] J. Ortega-Morla, A. Leis, A. Mallo, L. Morn-Fernndez, S. Guerreiro, A. Paz-Lpez, B. P©rez-Snchez, N. Snchez-Maro±o, A. Rodrguez-Arias, O. Fontenla-Romero, and F. Bellas, "Progtutor: A robotic-based framework to support teaching and learning of programming fundamentals," *IEEE Transactions on Learning Technologies*, vol. 18, pp. 783–797, 2025. [Online]. Available: https://doi.org/10.1109/TLT.2025.3598041

[10] S. Marwan, B. Akram, T. Barnes, and T. W. Price, "Adaptive immediate feedback for block-based programming: Design and evaluation," *IEEE Transactions on Learning Technologies*, vol. 15, no. 3, pp. 406–420, 2022. [Online]. Available: https://doi.org/10.1109/TLT.2022.3180984